Lab Exercise 8: Longest Substring Without Repeating Characters

Objective: To implement a linear-time optimization program finding the longest unique substring using the Sliding Window technique.

Aim: To compute lengths of unique string sequences in $O(n)$ time complexity instead of nested $O(n^2)$ brute-force paths.

Theory: A sliding window uses two pointer positions (`left` and `right`) to mark the boundary limits of a processing area. As the `right` pointer pulls in new data points, any duplicates trigger an internal update that moves the `left` boundary forward, preventing redundant calculations.

Algorithm / Procedure:

1. Input: A single string array sequence S.
2. Maintain a tracking layout (like a map or integer array index position ledger) recording character coordinates.
3. Advance the `right` pointer over the string. If a character is repeated within the current boundary range, pull the `left` pointer up behind its previous occurrence.
4. Update a global max variable tracking the window's total width: `maxLen = max(maxLen, right - left + 1)`.

Program Code: Problem4_LongestSubstring.java

```
import java.util.*;


public class Problem4_LongestSubstring {

   public static int findLongestUniqueSubstring(String s) {

      // TODO: Implement linear sliding window logic here

      return 0; // placeholder

   }


   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter the input string: ");

      String input = sc.next();
```

```
        int answer = findLongestUniqueSubstring(input);
        System.out.println("Length of longest unique substring: " + answer);
        sc.close();
    }
}
```

Sample Input / Output:

Input:

abcabcbb

Output:

Length of longest unique substring: 3

Evaluation Criteria (Total Marks: 10):

- Program compiles and runs successfully (2 Marks)
- Implementing true $O(n)$ sliding window loop design (3 Marks)
- Efficient tracking of character frequency states (2 Marks)
- Handling empty string edge cases (2 Marks)
- Adherence to clean variable naming conventions (1 Mark)